

Introduction

Data visualization is the process of representing information using graphical elements such as charts and plots. It helps users understand patterns, trends, comparisons, and relationships in data more effectively.

Visualization is widely used in Data Science, Machine Learning, Business Analytics, Research, and Dashboard Preparation.

This documentation explains two important Python visualization libraries:

- Matplotlib
- Pandas Visualization

2. Matplotlib Documentation

Matplotlib is one of the most popular Python libraries for creating static and interactive visualizations. It provides detailed customization and supports many graph types.

Features of Matplotlib

- Supports multiple graph types
- Highly customizable
- Works with NumPy and Pandas
- Suitable for scientific visualization
- Supports subplots and multiple figures

Advantages

- Detailed graph customization
- Large community support
- Professional-quality visualization

Limitations

- Requires more code
- Customization can be difficult for beginners

Installation

```
pip install matplotlib
```

2.1 Line Plot

Description:

The `plot()` function is used to create a line graph by connecting data points using straight lines. The `x` variable represents values on the horizontal axis and `y` represents values on the vertical axis. `title()` is used to give a heading to the graph and `show()` displays the output graph.

Line plots are commonly used when data changes continuously over time and help identify increasing or decreasing trends clearly.

Definition: A line plot connects data points using lines to show trends.

Real-World Use Case: Sales growth and stock market analysis.

Code Example:

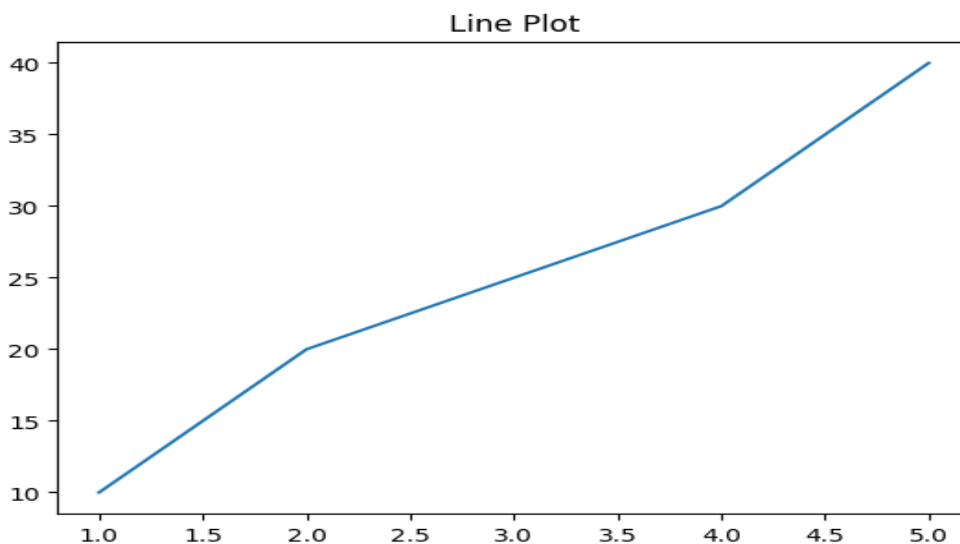
```
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [10,20,25,30,40]

plt.plot(x, y)
plt.title("Line Plot")
plt.show()
```

Output Explanation: The line graph displays trends and changes in values over time using connected data points.

Output:



2.2 Bar Chart

Description:

Rectangular bars are used in bar charts where the height of each bar represents the value of a category. `bar()` is used for plotting bar graphs with parameters such as x, y, width, and color. The x variable contains categories while y contains numerical values.

Bar charts make comparison between categories easier because the height of each bar represents the value of that category.

****Definition:****

Bar charts compare categories using rectangular bars.

Real-World Use Case: Student marks and product sales comparison.

Code Example:

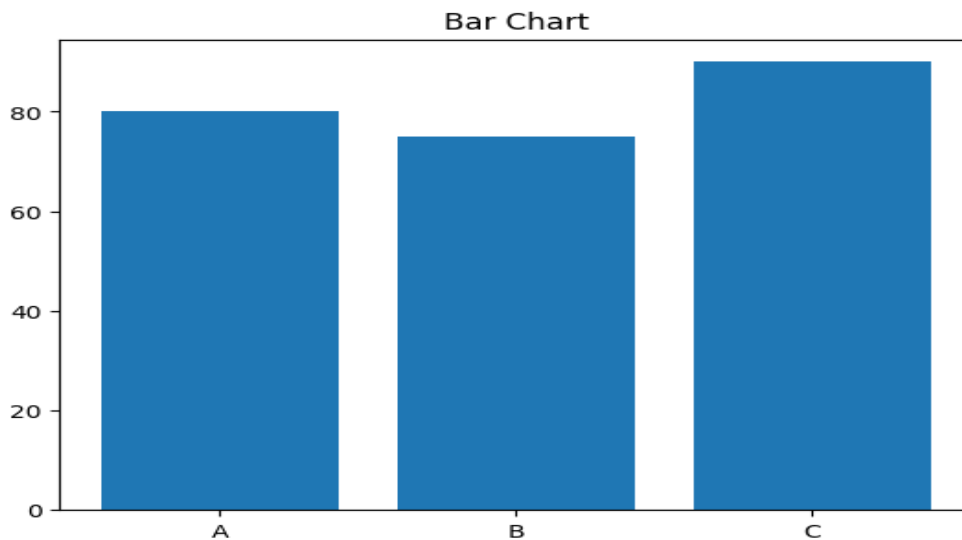
```
import matplotlib.pyplot as plt

students = ["A", "B", "C"]
marks = [80, 75, 90]

plt.bar(students, marks)
plt.title("Bar Chart")
plt.show()
```

Output Explanation: The bar graph compares categories where bar height represents the value of each category.

Output:



2.3 Scatter Plot

Description:

Scatter plots use individual points to represent relationships between two variables. `scatter()` is used to create the graph where x and y represent coordinates of points. This graph helps identify trends, patterns, and correlations between datasets.

Scatter plots help identify patterns, relationships, and correlations between two variables in a dataset.

Definition: Scatter plots show relationships between variables.

Real-World Use Case: Height vs weight analysis.

Code Example:

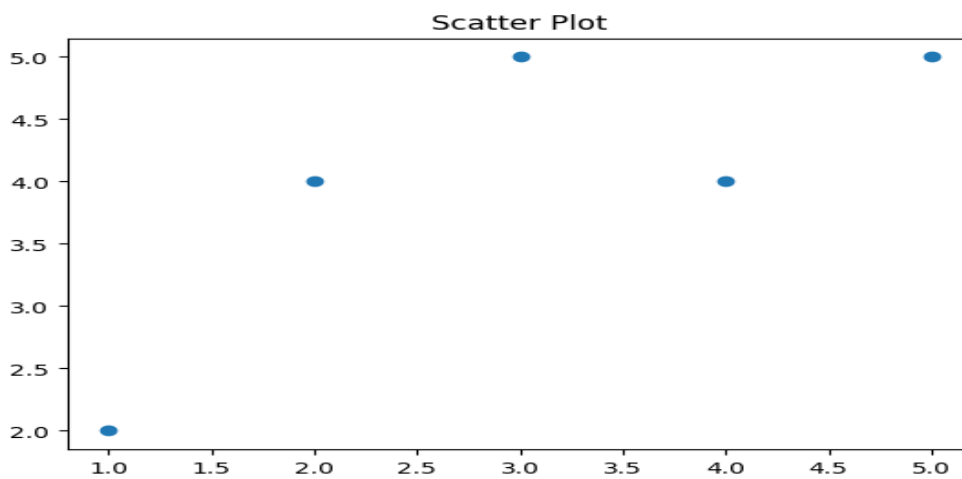
```
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [2,4,5,4,5]

plt.scatter(x, y)
plt.title("Scatter Plot")
plt.show()
```

Output Explanation: The scatter plot shows relationships and patterns between two variables using points.

Output:



2.4 Histogram

Description:

Histograms are used to represent frequency distribution of numerical data. `hist()` divides data into intervals called bins. The `bins` parameter controls the number of intervals shown in the graph.

Histograms group data into ranges called bins, helping users understand frequency distribution and data spread.

Definition: Histograms display frequency distribution of numerical data.

Real-World Use Case: Exam score analysis.

Code Example:

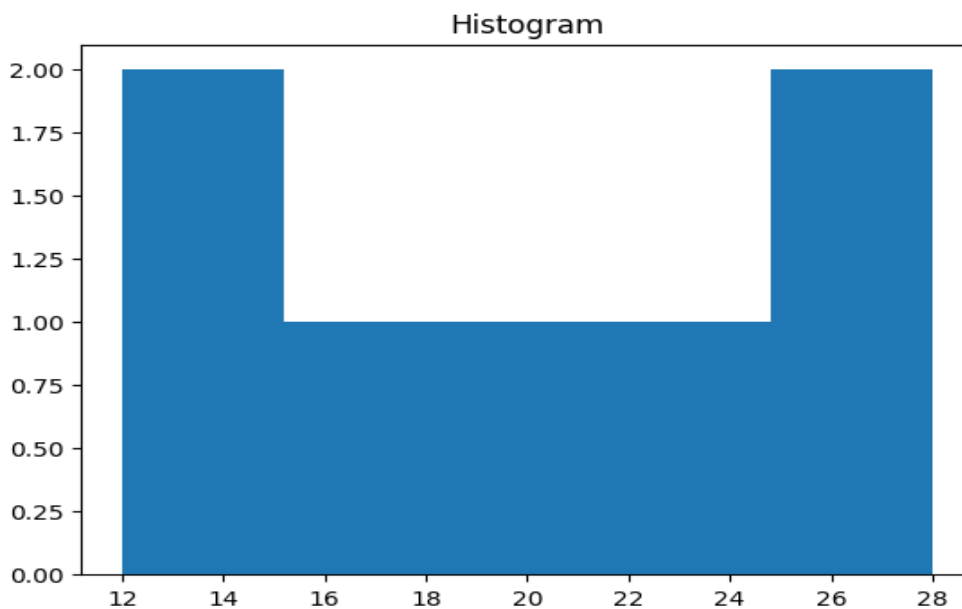
```
import matplotlib.pyplot as plt

data = [12,15,18,20,22,25,28]

plt.hist(data, bins=5)
plt.title("Histogram")
plt.show()
```

Output Explanation: The histogram groups data into intervals to show frequency distribution clearly.

Output:



5 Pie Chart

Description:

Pie charts represent data as slices of a circle where each slice shows percentage contribution. `pie()` is used for creating pie charts and labels are used to identify categories. `autopct` displays percentage values on the chart.

Pie charts visually represent percentages and are useful when showing parts of a whole dataset.

Definition: Pie charts represent percentage contribution.

Real-World Use Case: Budget and market share analysis.

Code Example:

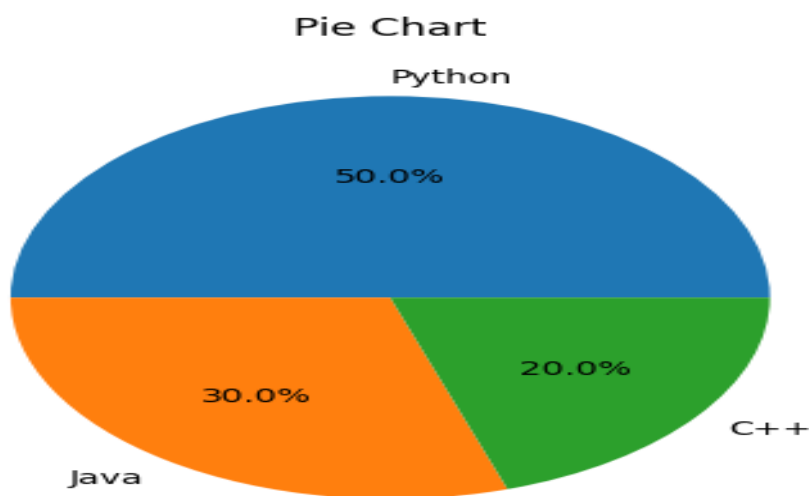
```
import matplotlib.pyplot as plt

sizes = [50,30,20]
labels = ["Python","Java","C++"]

plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Pie Chart")
plt.show()
```

Output Explanation: The pie chart represents percentage contribution of categories using circular slices.

Output:



3. Pandas Visualization Documentation

Pandas provides built-in visualization functions for creating graphs directly from DataFrames and Series. It is beginner-friendly and helps users quickly visualize data using simple syntax.

Pandas uses Matplotlib internally for generating plots, which allows easy integration with advanced customization features.

It is commonly used in:

- Data analysis
- Business reporting
- Machine learning
- Research projects

Pandas supports different graph types such as line plots, bar charts, histograms, scatter plots, pie charts, and box plots. These graphs help users understand trends, comparisons, and data distribution more effectively.

Features of Pandas Visualization

- Easy plotting syntax
- Direct DataFrame integration
- Quick graph generation
- Beginner friendly

Advantages

- Simple to use
- Requires less code
- Good for quick analysis

Limitations

- Less customizable
- Limited advanced visualization features

Installation

```
pip install pandas
```

3.1 Line Plot

Description:

The `plot()` function with `kind='line'` creates a line graph directly from the `DataFrame`. `x` specifies values for the horizontal axis and `y` specifies values for the vertical axis.

These plots are useful for analyzing trends and changes in sequential or time-based data.

Definition: Line plots display trends over time.

Real-World Use Case: Tracking daily sales.

Code Example:

```
import pandas as pd
import matplotlib.pyplot as plt

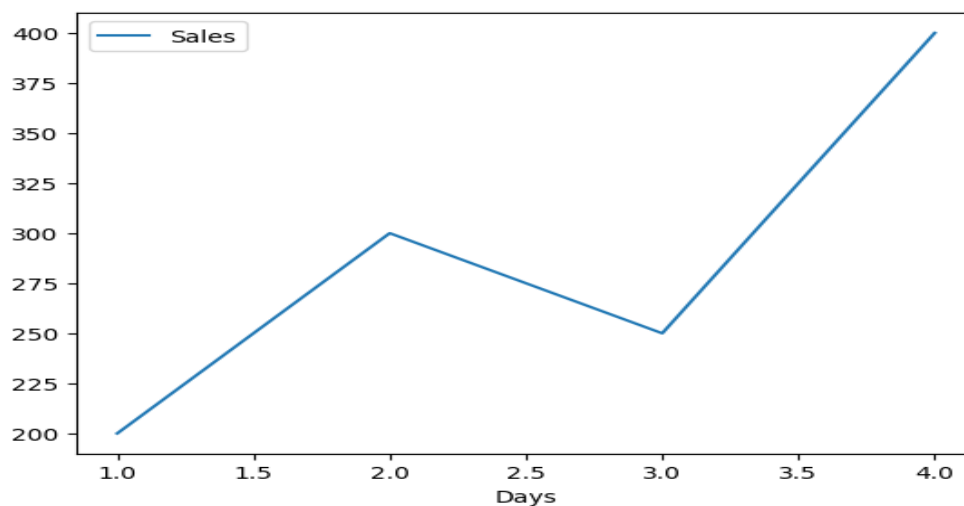
data = {
    "Days": [1, 2, 3, 4],
    "Sales": [200, 300, 250, 400]
}

df = pd.DataFrame(data)

df.plot(x="Days", y="Sales", kind="line")
plt.show()
```

Output Explanation: The line graph displays continuous trends directly from the `DataFrame` values.

Output:



3.2 Bar Plot

Description:

Bar plots compare categories using rectangular bars. `kind='bar'` is used to generate the graph where category names are placed on the x-axis and values are represented by bar height.

Bar plots provide a simple way to compare different categories and their numerical values.

Definition: Bar plots compare category values.

Real-World Use Case: Fruit quantity comparison.

Code Example:

```
import pandas as pd
import matplotlib.pyplot as plt

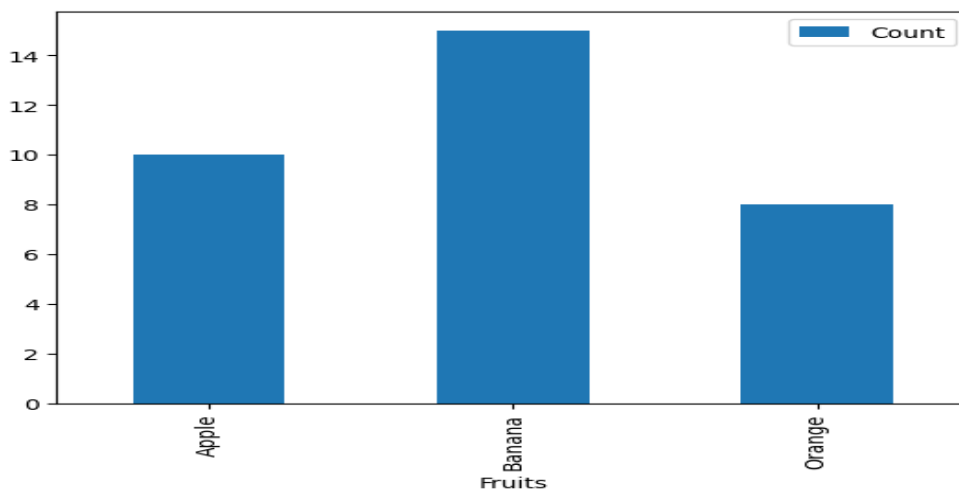
data = {
    "Fruits": ["Apple", "Banana", "Orange"],
    "Count": [10, 15, 8]
}

df = pd.DataFrame(data)

df.plot(x="Fruits", y="Count", kind="bar")
plt.show()
```

Output Explanation: The bar plot compares category values where each bar represents one category.

Output:



3.3 Histogram

Description:

The histogram groups numerical values into ranges to show data distribution. `kind='hist'` is used to create histogram graphs from DataFrame columns.

Histograms help analyze how data values are distributed across different intervals.

Definition: Histograms show distribution of data.

Real-World Use Case: Marks distribution analysis.

Code Example:

```
import pandas as pd
import matplotlib.pyplot as plt

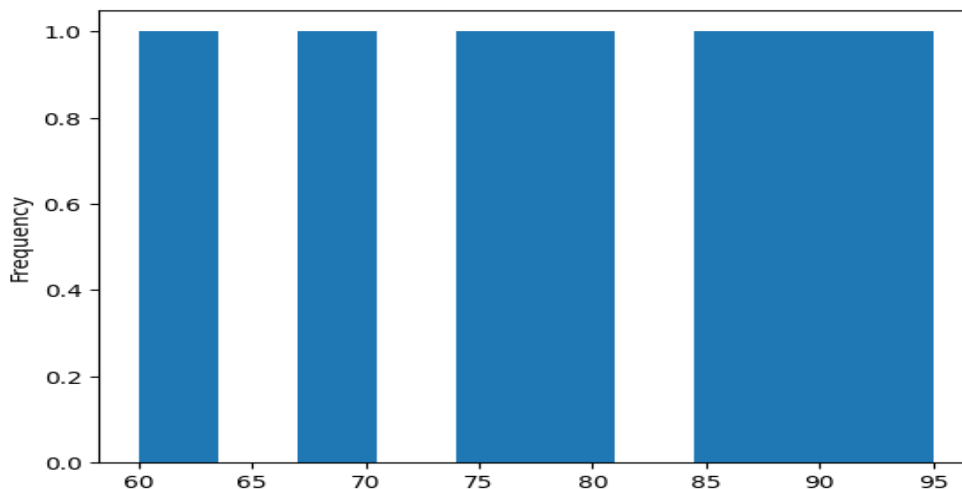
data = {
    "Marks": [60, 70, 75, 80, 85, 90, 95]
}

df = pd.DataFrame(data)

df["Marks"].plot(kind="hist")
plt.show()
```

Output Explanation: The histogram shows how numerical values are distributed across ranges.

Output:



3.4 Scatter Plot

Description:

Scatter plots display relationships between two variables using points. `kind='scatter'` creates the graph where each point represents one observation from the dataset.

Scatter plots are useful for understanding relationships and identifying clusters or outliers in data.

Definition: Scatter plots show relationships between variables.

Real-World Use Case: Study hours vs marks.

Code Example:

```
import pandas as pd
import matplotlib.pyplot as plt

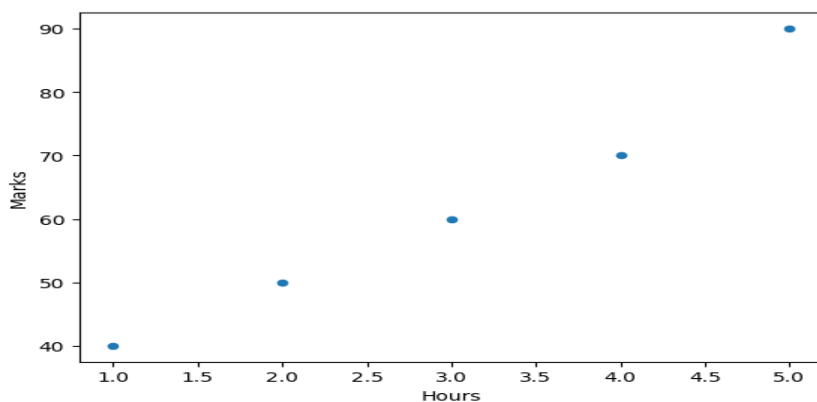
data = {
    "Hours": [1, 2, 3, 4, 5],
    "Marks": [40, 50, 60, 70, 90]
}

df = pd.DataFrame(data)

df.plot(x="Hours", y="Marks", kind="scatter")
plt.show()
```

Output Explanation: The scatter plot helps identify relationships between study hours and marks.

Output:



3.5 Pie Chart

Description:

Pie charts display percentage contribution of categories. `kind='pie'` creates the chart and `autopct` is used to display percentage values on each slice.

Pie charts display percentage contribution clearly, making comparison between categories easier.

Definition: Pie charts show percentage contribution.

Real-World Use Case: Market share and budget analysis.

Code Example:

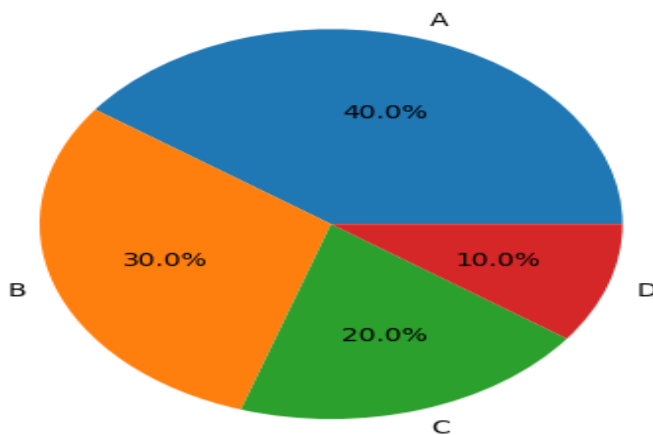
```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.Series(
    [40,30,20,10],
    index=["A","B","C","D"]
)

data.plot(kind="pie", autopct='%1.1f%%')
plt.show()
```

Output Explanation: The pie chart visually displays percentage contribution of each category.

Output:



COMPARISON OF MATPLOTLIB AND PANDAS VISUALIZATION

Matplotlib

Core Library

Matplotlib is a comprehensive Python visualization library used for creating static, animated, and interactive graphs. It provides detailed control over every part of a figure and is widely used in data science and research.

Flexibility

Matplotlib supports many graph types, including line plots, scatter plots, bar charts, histograms, pie charts, box plots, area plots, and subplots. Users can customize graphs according to their requirements.

Customization

Matplotlib provides extensive customization features such as:

- Colors
- Line styles
- Fonts
- Labels
- Legends
- Markers
- Figure size

Integration

Matplotlib works well with libraries like:

- NumPy
- Pandas
- SciPy

This makes it useful for scientific computing and data analysis projects.

Advantages of Matplotlib

- High customization and control
- Supports many graph types
- Professional-quality visualizations
- Large community support
- Suitable for advanced visualization tasks

Limitations of Matplotlib

- Requires more code
- Slightly difficult for beginners
- Limited built-in interactivity

Pandas Visualization

Built-in Visualization

Pandas provides built-in plotting functions that help users create graphs directly from DataFrames and Series. It uses Matplotlib internally for rendering visualizations.

Simplicity

Pandas visualization is beginner-friendly because graphs can be created using simple syntax with minimal code.

DataFrame Integration

Pandas visualization works directly with structured datasets, making it useful for quick data analysis and reporting.

Supported Graph Types

Pandas supports:

- Line plots
- Bar plots
- Histograms
- Scatter plots
- Pie charts

- Box plots
- Area plots

Ease of Use

Users can generate graphs directly from datasets without performing complex plotting steps manually.

Advantages of Pandas Visualization

- Easy and beginner friendly
- Requires less coding
- Quick visualization generation
- Direct integration with DataFrames
- Good for exploratory data analysis

Limitations of Pandas Visualization

- Less customizable compared to Matplotlib
- Limited advanced plotting features
- Depends on Matplotlib internally

Overall Comparison

The choice between Matplotlib and Pandas Visualization depends on the project requirements.

- Matplotlib is preferred when detailed customization and professional-quality visualizations are needed.
- Pandas Visualization is preferred for quick and simple plotting directly from datasets.

Many users combine both libraries together, using Pandas for quick analysis and Matplotlib for advanced customization.